Module 3 – Frontend – CSS and CSS3

CSS Selectors & Styling

• Question 1: What is a CSS selector? Provide examples of element, class, and ID selectors.

Ans:

A CSS selector is a pattern used to select and target HTML elements that you want to style. Selectors act as bridges between your HTML structure and CSS rules, allowing you to apply specific styles to particular elements.

1. Element Selector

Element selectors target HTML elements by their tag name. They're the most basic type of selector.

```
p{
    color: blue;
}
```

This applies blue color to all <p> tags.

2. Class Selector

Class selectors target elements with a specific class attribute. They're reusable and can be applied to multiple elements.
A class selector uses a . (dot).

```
.title {
    color: red;
}
```

HTML:

```
<h1 class="title">Hello</h1>
```

3. ID Selector

ID selectors target elements with a specific id attribute. IDs must be unique within a document.
An ID selector uses a # symbol.

```
#header {
   background-color: yellow;
}
```

HTML:

```
<div id="header">Welcome</div>
```

• Question 2: Explain the concept of CSS specificity. How do conflicts between multiple styles get resolved?

Ans:

CSS Specificity

CSS specificity is a set of rules used by the browser to decide which CSS style will be applied to an HTML element when more than one style rule targets the same element. Since multiple CSS selectors can affect the same element, the browser must determine which rule has the highest priority. This priority system is called **specificity**.

Specificity is one of the most important concepts in CSS because it controls how styles are applied and how conflicts between different CSS rules are resolved.

Why CSS Specificity Is Needed

In a webpage, developers often write many CSS rules for the same element. For example, an element may receive styles from:

- Element selectors
- Class selectors
- ID selectors
- Inline CSS
- External stylesheets

- Internal stylesheets

Without specificity, the browser would not know which style should finally appear on the webpage.

Specificity creates a hierarchy of priority so the browser can correctly choose the final style.

How CSS Conflicts Are Resolved

When multiple CSS rules apply to the same element, conflicts are resolved using the following order:

1. Importance (!important)

The !important rule has the highest priority.

Example:

```
p {
   color: blue !important;
}

#demo {
   color: red;
}
```

Even though the ID selector has higher specificity, the blue color is applied because of !important.

The !important keyword should be used carefully because excessive use makes CSS difficult to manage.

2. Inline CSS

Inline styles override internal and external CSS.

Example:

```
<p style="color: orange;" id="demo">Hello</p>
#demo {
   color: red;
}
```

The text becomes orange because inline CSS has higher priority.

3. Higher Specificity Selector Wins

Example:

```
p {
    color: blue;
}

.text {
    color: green;
}

#title {
    color: red;
}
```

The ID selector wins because it has the highest specificity.

4. Last Rule Overrides Earlier Rule

If two selectors have equal specificity, the rule written later is applied.

Example:

```
p {
    color: blue;
}

p {
    color: green;
}
```

Both selectors have equal specificity, so the second rule is applied.

Result: Green color.

• Question 3: What is the difference between internal, external, and inline CSS? Discuss the advantages and disadvantages of each approach.

Ans:

Difference Between Internal, External, and Inline CSS

CSS (Cascading Style Sheets) is used to control the appearance and layout of web pages. It allows developers to apply styles such as colors, fonts, spacing, borders, animations, and layouts to HTML elements.

CSS can be added to HTML documents in three different ways:

1. Inline CSS
2. Internal CSS
3. External CSS

Each method has its own purpose, advantages, and disadvantages. Choosing the correct approach depends on the size, complexity, and requirements of the website.

1. Inline CSS

Definition

Inline CSS is a method of applying CSS directly inside an HTML element using the `style` attribute.

The style affects only that specific element.

Syntax

```
<p style="color: red; font-size: 20px;">
   Hello World
</p>
```

In this example:

- `color: red` changes the text color.
- `font-size: 20px` changes the font size.

The styles are written directly inside the HTML tag.

Features of Inline CSS

- Applied to a single element only.
- Written inside the HTML tag.
- Has very high specificity.
- Overrides internal and external CSS in most cases.

Advantages of Inline CSS

1. Quick and Simple

2. No Separate CSS File Needed

3. High Priority

4. Useful for Dynamic Styling

Disadvantages of Inline CSS

1. Poor Maintainability

2. Repetition of Code

3. Mixes Content and Presentation

4. Not Reusable

5. Difficult for Large Projects

2. Internal CSS

Definition

Internal CSS is written inside the <style> tag within the <head> section of an HTML document.

The styles apply only to that particular webpage.

Syntax

```
<head>
   <style>
      p {
         color: blue;
         font-size: 18px;
      }
   </style>
</head>
```

This style affects all <p> elements in the same page.

Features of Internal CSS

- Written inside the same HTML file.

- Uses the <style> tag.
- Styles apply only to one webpage.
- Better organized than inline CSS.
-

Advantages of Internal CSS

1. Better Organization

2. No External File Required

3. Reusable Within Same Page

4. Easier Maintenance Than Inline CSS

5. Faster Loading for Single Pages

Disadvantages of Internal CSS

1. Cannot Be Reused Across Multiple Pages

2. Increases HTML File Size

3. Difficult to Maintain Large Projects

4. No Browser Caching

3. External CSS

Definition

External CSS is written in a separate .css file and connected to the HTML document using the <link> tag.

This is the most professional and widely used CSS method.

Syntax

HTML File

```
<head>
   <link rel="stylesheet" href="style.css">
</head>
```

CSS File (style.css)

```
p {
   color: green;
   font-size: 20px;
}
```

Features of External CSS

- CSS is stored in a separate file.
- One CSS file can style multiple webpages.
- Provides maximum reusability and maintainability.

Advantages of External CSS

1. Best Code Organization

2. Reusable Across Multiple Pages

3. Easy Maintenance

4. Reduces Code Duplication

5. Faster Website Performance

6. Best for Large Projects

7. Better Team Collaboration

Disadvantages of External CSS

1. Requires Additional File

2. Initial Loading Delay

3. Link Errors May Break Styling

CSS Box Model

• Question 1: Explain the CSS box model and its components (content, padding, border, margin). How does each affect the size of an element?

Ans:

The **CSS Box Model** is a fundamental concept in CSS that describes how every HTML element is represented as a rectangular box on a webpage. It determines the total size, spacing, and layout of elements.

Every element in HTML is considered a box consisting of four main parts:

1. Content
2. Padding
3. Border
4. Margin

These components together decide the final size and spacing of the element.

Structure of the CSS Box Model

```
Margin
 └── Border
     └── Padding
         └── Content
```

1. Content

The **content area** is the main part of the element where text, images, or other HTML content appears.

The width and height properties normally apply only to the content area.

Example:

```
div {
   width: 200px;
   height: 100px;
}
```

Here:

- Content width = 200px
- Content height = 100px

The content area does not include padding, border, or margin.

2. Padding

Padding is the space between the content and the border. It creates inner spacing inside the element.

Example:

```
div {
   padding: 20px;
}
```

This adds 20px space inside the element around the content.

Effects of Padding

- Increases the total size of the element.
- Background color is visible in the padding area.
- Makes content look less crowded.

3. Border

The border surrounds the padding and content.

Example:

```
div {
   border: 5px solid black;
}
```

This creates a 5px black border around the element.

Effects of Border

- Adds extra thickness to the element size.
- Separates the element visually from other elements.
- Comes outside the padding.

4. Margin

Margin is the outer space outside the border. It creates distance between elements.

Example:

```
div {
   margin: 30px;
}
```

This adds 30px space outside the element.

Effects of Margin

- Creates spacing between neighboring elements.
- Does not affect background color.
- Increases total occupied space on the webpage.

• Question 2: What is the difference between border-box and content-box box-sizing in CSS? Which is the default?

Ans:

The box-sizing property in CSS defines how the width and height of an element are calculated.

There are two main values:

1. content-box
2. border-box

1. content-box

content-box is the default box-sizing value in CSS.

In this model:

- Width and height apply only to the content area.
- Padding and border are added outside the specified width and height.
-

Example

```
div {
   width: 200px;
   padding: 20px;
   border: 5px solid black;
   box-sizing: content-box;
}
```

Actual Width Calculation

200px (content)
+ 40px padding

+ 10px border
= 250px total width

So the final size becomes larger than the specified width.

Advantages of content-box

- Traditional CSS behavior
- Gives separate control over content size
- Useful in some detailed layout designs

Disadvantages of content-box

- Total size becomes difficult to calculate
- Layouts may break easily
- Responsive design becomes harder

2. border-box

In border-box, padding and border are included inside the specified width and height.

The total size remains fixed.

Example

```
div {
   width: 200px;
   padding: 20px;
   border: 5px solid black;
   box-sizing: border-box;
}
```

Actual Width

Total width remains 200px

The content area automatically shrinks to fit padding and border inside 200px.

Advantages of border-box

- Easier size calculation
- Better for responsive layouts
- Prevents unexpected size increases
- Widely used in modern web development

Disadvantages of border-box

- Content area becomes smaller when padding increases
- Slightly different from traditional CSS behavior

CSS Flexbox

• Question 1: What is CSS Flexbox, and how is it useful for layout design? Explain the terms flex-container and flex-item.

Ans:

CSS Flexbox (Flexible Box Layout) is a modern CSS layout model used to arrange, align, and distribute elements efficiently inside a container. It helps developers create flexible and responsive layouts without using complex positioning or floating techniques.

Flexbox is mainly designed for one-dimensional layouts, meaning it can arrange elements either in:

- a row (horizontal direction), or
- a column (vertical direction).

It makes it easier to:

- align elements,
- create responsive designs,
- distribute space evenly,
- center content,
- and control element order and size.

Before Flexbox, developers commonly used:

- floats,
- tables,
- positioning,
  which were more difficult and less flexible for responsive layouts.

Basic Structure of Flexbox

Flexbox works using two important components:

1. Flex Container
2. Flex Items

1. Flex Container

A flex container is the parent element that holds flex items.

To make an element a flex container, the `display: flex;` property is used.

Example:

```
.container {
   display: flex;
}
```

Here, `.container` becomes a flex container.

The flex container controls:

- direction of items,
- alignment,
- spacing,
- wrapping behavior,
- and positioning of child elements.

2. Flex Items

Flex items are the child elements inside a flex container.

Example:

```
<div class="container">
   <div>Item 1</div>
   <div>Item 2</div>
   <div>Item 3</div>
</div>
```

All three `<div>` elements become flex items because they are inside the flex container.

Flex items automatically arrange themselves according to the flexbox properties applied to the container.

How Flexbox Is Useful for Layout Design

Flexbox is very useful in modern web development because it simplifies layout creation.

1. Easy Alignment

Flexbox makes horizontal and vertical alignment simple.

Example:

- Centering text or buttons
- Aligning navigation menus

2. Responsive Design

Flexbox automatically adjusts element sizes and spacing for different screen sizes.

This is useful for:

- mobile layouts,
- tablets,
- desktop screens.

3. Space Distribution

Flexbox can distribute extra space evenly between elements.

Example:

- equal-width columns,
- balanced card layouts.

4. Flexible Layout Direction

Elements can be arranged:

- horizontally,
- vertically,
- reversed,
  without changing HTML structure.

5. Reduces Complex CSS

Flexbox removes the need for:

- floats,
- clearfix hacks,
- complicated positioning.

This makes CSS cleaner and easier to maintain.

Example of Flexbox

```
.container {
   display: flex;
}
```

This places all child elements in a horizontal row by default.

Advantages of Flexbox

- Easy alignment and spacing
- Better responsive layouts
- Cleaner code
- Flexible item sizing
- Easy centering
- Less dependency on floats

Disadvantages of Flexbox

- Mainly one-dimensional
- Complex layouts may still require CSS Grid
- Older browser support was limited (mostly solved now)

• Question 2: Describe the properties justify-content, align-items, and flexdirection used in Flexbox.

Ans:

These are important Flexbox properties used to control the arrangement and alignment of flex items.

1. flex-direction

The flex-direction property defines the direction in which flex items are placed inside the flex container.

Syntax

```
.container {
   flex-direction: row;
}
```

Values of flex-direction

1. row

Items are arranged horizontally from left to right.

flex-direction: row;

This is the default value.

2. row-reverse

Items are arranged horizontally from right to left.

flex-direction: row-reverse;

3. column

Items are arranged vertically from top to bottom.

flex-direction: column;

4. column-reverse

Items are arranged vertically from bottom to top.

flex-direction: column-reverse;

Importance of flex-direction

It controls the main axis of the flex container and changes how items flow inside the layout.

2. justify-content

The justify-content property controls the alignment of flex items along the main axis.

The main axis depends on flex-direction.

- In row, the main axis is horizontal.
- In column, the main axis is vertical.

Syntax

```
.container {
    justify-content: center;
}
```

Common Values of justify-content

1. flex-start

Items align at the beginning of the container.

justify-content: flex-start;

Default value.

---

2. flex-end

Items align at the end of the container.

justify-content: flex-end;

3. center

Items align at the center.

justify-content: center;

4. space-between

Equal space is placed between items.

justify-content: space-between;

First and last items touch container edges.

5. space-around

Equal space is added around each item.

justify-content: space-around;

6. space-evenly

Equal spacing appears everywhere.

justify-content: space-evenly;

Importance of justify-content

It helps distribute extra space and align items properly along the main axis.

3. align-items

The align-items property controls alignment along the cross axis.

The cross axis is opposite to the main axis.

- If flex-direction: row, cross axis is vertical.
- If flex-direction: column, cross axis is horizontal.

Syntax

```
.container {
    align-items: center;
}
```

Common Values of align-items

1. stretch

Items stretch to fill the container.

align-items: stretch;

Default value.

2. flex-start

Items align at the top or start of the cross axis.

align-items: flex-start;

3. flex-end

Items align at the end of the cross axis.

align-items: flex-end;

4. center

Items align at the center of the cross axis.

align-items: center;

5. baseline

Items align according to text baseline.

align-items: baseline;

CSS Grid

• Question 1: Explain CSS Grid and how it differs from Flexbox. When would you use Grid over Flexbox?

Ans:

CSS Grid is a modern CSS layout system used to create structured and responsive layouts using rows and columns. It helps developers organize webpage content in a clean and flexible way. Grid is mainly used for designing full webpage layouts, dashboards, galleries, and complex sections.

To use Grid, the parent element is converted into a grid container using:

display: grid;

The child elements inside the container automatically become grid items.

CSS Grid works in two dimensions:

- rows
- columns

This means developers can control both horizontal and vertical layouts at the same time.

Example:

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
}
```

This creates three equal columns.

Flexbox and Grid are both layout systems, but they are designed for different purposes.

Flexbox is a one-dimensional layout system. It arranges items either in:

- a row
- or a column

Flexbox is best for:

- navigation bars
- buttons
- menus
- aligning small components

Example:

```
.container {
    display: flex;
}
```

Grid is a two-dimensional layout system. It manages rows and columns together, making it more suitable for complex layouts.

Grid is better for:

- webpage structures
- image galleries
- dashboard layouts
- portfolio sections

Another difference is layout control. Flexbox focuses more on alignment and spacing, while Grid provides better control over exact item placement.

Grid should be used when:

- both rows and columns are required
- layout structure is complex
- precise positioning is needed

Flexbox should be used when:

- content flows in one direction
- simple alignment is needed

- components are small

In modern web development, both Grid and Flexbox are often used together.

• Question 2: Describe the grid-template-columns, grid-template-rows, and grid-gap properties. Provide examples of how to use them.

Ans:

The grid-template-columns property defines the number and size of columns in a grid layout.

Syntax:

```
grid-template-columns: values;
```

Example:

```
.container {
    display: grid;
    grid-template-columns: 200px 200px 200px;
}
```

This creates three columns, each 200px wide.

The fr unit can also be used to divide available space equally.

Example:

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
}
```

This creates three equal columns.

Another example:

```
grid-template-columns: 1fr 2fr 1fr;
```

The middle column becomes twice as large as the other columns.

The grid-template-rows property defines the number and height of rows.

Syntax:

```
grid-template-rows: values;
```

Example:

```
.container {
  display: grid;
  grid-template-rows: 100px 200px;
}
```

This creates:

- first row = 100px
- second row = 200px

Fraction units can also be used:

```
grid-template-rows: 1fr 2fr;
```

The second row becomes twice as large as the first row.

The grid-gap property adds spacing between rows and columns in a grid layout.

Syntax:

```
grid-gap: 20px;
```

Example:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-gap: 20px;
}
```

This adds 20px spacing between all grid items.

Different row and column gaps can also be added:

```
grid-gap: 10px 20px;
```

Here:

- row gap = 10px
- column gap = 20px

Modern CSS commonly uses the gap property instead of grid-gap.

Example:

```
gap: 20px;
```

Combined Example:

```
.container {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 100px 100px;
    gap: 15px;
}
```

This creates:

- three columns
- two rows
- 15px spacing between items

CSS Grid is powerful because it allows developers to create responsive and organized layouts with less code and better control over webpage structure.

Responsive Web Design with Media Queries

• Question 1: What are media queries in CSS, and why are they important for responsive design?

Ans:

Media queries are a feature in CSS used to apply different styles to a webpage based on the screen size, device type, resolution, or orientation.

They are mainly used in responsive web design to make websites look good on:

- mobile phones
- tablets
- laptops

- desktop screens

Media queries allow developers to change the layout and styling depending on the device screen size.

A media query starts with the @media rule.

Basic syntax:

```
@media condition {
   CSS rules
}
```

Example:

```
@media (max-width: 600px) {
   body {
      background-color: lightblue;
   }
}
```

In this example:

- when the screen width becomes 600px or smaller,
- the background color changes to light blue.

Media queries are important because modern users access websites from many different devices with different screen sizes. Without responsive design, websites may:

- look too large on mobile devices,
- have broken layouts,
- require horizontal scrolling,
- or become difficult to use.

Media queries help solve these problems by adjusting:

- font sizes,
- layouts,
- images,
- navigation menus,
- spacing,
- and element positioning.

They improve:

- user experience,
- readability,
- accessibility,
- and mobile compatibility.

Media queries are one of the most important parts of responsive web development.

Common media query conditions include:

max-width
min-width
orientation
max-height
min-height

Example of tablet styling:

```
@media (max-width: 768px) {
    .container {
        width: 100%;
    }
}
```

This changes the container width for smaller devices.

Responsive design using media queries is important because:

- mobile usage is very high,
- websites must work on all devices,
- search engines prefer mobile-friendly websites,
- and responsive websites provide a better user experience.

• Question 2: Write a basic media query that adjusts the font size of a webpage for screens smaller than 600px

Ans:

Example:

```
body {
    font-size: 20px;
}

@media (max-width: 600px) {
    body {
        font-size: 14px;
    }
}
```

Explanation:

- The default font size is 20px.
- When the screen width becomes 600px or smaller,

- the font size changes to 14px.

This helps improve readability on smaller screens like mobile phones.

Another example using a heading:

```
h1 {
   font-size: 40px;
}

@media (max-width: 600px) {
   h1 {
      font-size: 24px;
   }
}
```

In this example:

- large screens show a bigger heading,
- smaller screens show a smaller heading for better responsiveness.

Media queries make websites flexible and adaptable across different screen sizes and devices.

Typography and Web Fonts

• Question 1: Explain the difference between web-safe fonts and custom web fonts. Why might you use a web-safe font over a custom font?

Ans:

Fonts are an important part of web design because they affect the appearance, readability, and user experience of a webpage. In CSS, developers can use different types of fonts to style text. Two common types are:

- web-safe fonts
- custom web fonts

Web-safe fonts are fonts that are already installed on most operating systems and devices. Since these fonts are available on almost all computers, browsers can display them without downloading additional font files.

Examples of web-safe fonts include:

- Arial
- Times New Roman
- Verdana
- Georgia
- Courier New

Example:

```
p {
   font-family: Arial, sans-serif;
}
```

In this example, the browser uses Arial if it is available.

Web-safe fonts are considered reliable because they work consistently across different browsers and devices.

Custom web fonts are fonts that are not usually installed on the user's system. These fonts are downloaded from external files or font services when the webpage loads.

Custom fonts allow developers to create unique and attractive website designs.

Popular sources of custom fonts include:

- Google Fonts
- Adobe Fonts
- self-hosted font files

Example:

```
h1 {
   font-family: 'Poppins', sans-serif;
}
```

If the user does not already have the font installed, the browser downloads it from the web.

The main difference between web-safe fonts and custom web fonts is availability.

Web-safe fonts:

- already exist on the user's device
- do not require downloading
- load faster
- provide better compatibility

Custom web fonts:

- require downloading font files
- provide more design flexibility

- improve visual appearance
- allow unique typography

Developers may choose web-safe fonts over custom fonts for several reasons.

First, web-safe fonts improve website performance because no additional font files need to be downloaded. Faster loading times improve user experience and website speed.

Second, web-safe fonts offer better compatibility across browsers and operating systems. Since the fonts are already installed, there is less risk of display problems.

Third, web-safe fonts are useful for simple or professional websites where readability and performance are more important than decorative design.

However, custom web fonts are preferred when branding, modern design, and unique appearance are important.

• Question 2: What is the font-family property in CSS? How do you apply a custom Google Font to a webpage?

Ans:

The font-family property in CSS is used to define the typeface or font used for text in an HTML element.

Syntax:

```
font-family: font-name;
```

Example:

```
p {
   font-family: Arial, sans-serif;
}
```

In this example:

- Arial is the primary font.
- sans-serif is the fallback font family.

Fallback fonts are important because if the first font is unavailable, the browser uses the next available font.

The font-family property can contain:

- one font
- multiple fallback fonts
- generic font families

Common generic font families include:

- serif
- sans-serif
- monospace
- cursive
- fantasy

Example with multiple fonts:

```
body {
   font-family: "Poppins", Arial, sans-serif;
}
```

The browser first tries to use Poppins. If unavailable, it uses Arial, then a generic sans-serif font.

To apply a custom Google Font to a webpage, developers usually follow these steps.

Step 1: Visit Google Fonts and choose a font.

Popular Google Fonts:

- Poppins
- Roboto
- Open Sans
- Montserrat

Step 2: Copy the provided <link> tag and place it inside the <head> section of the HTML document.

Example:

```
<head>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap" rel="stylesheet">
</head>
```

This imports the Poppins font from Google Fonts.

Step 3: Use the font in CSS with the font-family property.

Example:

```
body {
   font-family: 'Poppins', sans-serif;
}
```

Now the webpage text uses the Poppins font.

Google Fonts are widely used because they:

- are free,
- easy to use,
- support many languages,
- and provide modern typography styles.

Using custom fonts improves the visual design and branding of websites, while the font-family property allows developers to control text appearance effectively.